

MOBILE_TESTING_GUIDE

NetworkPeer Mobile Testing Guide

Complete guide for testing the Android and iOS apps on Mac M1

Prerequisites

Mac M1 Environment Setup

```
# Java 17 (ARM64) - required for Android build
brew install openjdk@17
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/openjdk.jdk/Contents/Home"
export PATH="$JAVA_HOME/bin:$PATH"

# Verify
arch -arm64 java -version
# Should show: openjdk version "17.0.x" ... aarch64

# Android SDK
mkdir -p ~/Library/Android/sdk
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$ANDROID_HOME/platform-tools:$ANDROID_HOME/cmdline-
tools/latest/bin:$PATH"

# Install SDK components (if not already installed)
sdkmanager "platforms;android-36" "build-tools;35.0.0" "platform-
tools"
```

Option A: Physical Android Device (Recommended)

1. Enable Developer Options on Android

1. **Settings** → **About Phone** → **Software Information**
2. Tap **Build Number** 7 times → "You are now a developer!"

3. Settings → Developer Options → Enable **USB Debugging**

2. Connect to Mac

```
# Connect via USB-C
# Allow USB debugging on phone prompt

# Verify connection
adb devices
# Should show: <serial>    device
```

3. Build & Install

```
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/openjdk.jdk/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"

cd apps/android
./gradlew installDevelopmentDebug --no-daemon

# Or build APK first, then install:
./gradlew assembleDevelopmentDebug --no-daemon
adb install -r app/build/outputs/apk/development/debug/app-
development-debug.apk
```

4. Launch & Test

```
# Launch app
adb shell monkey -p com.networkpeer.mobile.dev -c
    android.intent.category.LAUNCHER 1

# View logs
adb logcat -s "NetworkPeer:*"

# Or filter by tag
adb logcat -s "NetworkPeerAPI" "NetworkPeerUI"
    "NetworkPeerCamera"
```

5. Screen Mirroring (for demo)

```
# Install scrcpy
brew install scrcpy

# Mirror phone screen to Mac
scrcpy --stay-awake --turn-screen-on --power-off-on-close=false
```

Option B: Android Emulator (No Physical Device)

1. Create ARM64 Emulator

```
# Open Android Studio → More Actions → Virtual Device Manager  
# Or use command line:  
avdmanager create avd -n "pixel7_arm64" -k "system-  
images;android-36;default;arm64-v8a" -d pixel_7
```

2. Launch Emulator

```
# With GPU acceleration for M1  
emulator -avd pixel7_arm64 -gpu host -no-audio -no-boot-anim  
  
# Wait for boot, then verify  
adb devices
```

3. Build & Install

```
cd apps/android  
./gradlew installDevelopmentDebug --no-daemon
```

4. Simulate Camera & Location

- **Camera:** Emulator sidebar → Camera → "Virtual Scene" or "Webcam0"
 - **Location:** Emulator sidebar → Location → Set GPS coordinates
-

Option C: iOS Testing (Mac M1 Only)

iOS Simulator

```
cd apps/ios  
open NetworkPeer.xcworkspace  
  
# In Xcode:  
# 1. Select "iPhone 15" or "iPhone 16" simulator  
# 2. Press Cmd+R to build and run
```

Physical iPhone

```
# 1. Connect iPhone via USB-C/Lightning  
# 2. Trust computer on iPhone  
# 3. In Xcode: Window → Devices and Simulators → select iPhone  
# 4. Set Signing Team: Project → Target → Signing & Capabilities  
→ Team: Your Apple ID
```

```
# 5. Press Cmd+R to deploy  
# 4. On iPhone: Settings → General → VPN & Device Management →  
Trust Developer Profile
```

API Configuration for Local Testing

The app points to production API by default. To test against local backend:

1. Start Local Backend

```
cd NetworkPeer-main  
npm install  
npm run dev # Runs on http://localhost:3000
```

2. Configure Android for Local API

```
# Edit apps/android/networkpeer.development.local.properties  
API_BASE_URL=http://10.0.2.2:3000/api/v1/  
STRIPE_PUBLISHABLE_KEY=pk_test_your_key  
REALTIME_URL=http://10.0.2.2:3000  
REALTIME_ORIGIN=
```

Note: `10.0.2.2` is the Android emulator's alias for the host machine's localhost.

3. Rebuild & Test

```
cd apps/android  
./gradlew assembleDevelopmentDebug --no-daemon  
adb install -r app/build/outputs/apk/development/debug/app-  
development-debug.apk
```

Test Scenarios

1. Post a Job Flow

1. Open app → Login/Register (phone + OTP)
2. Navigate to "Post a Job"
3. Complete 6-step wizard:
 - **Job Info:** Title, description, location
 - **Task Type:** Collection / Correction
 - **Media:** Photo/Video requirements
 - **Capacity:** Single / Capped / Unlimited
 - **Escrow:** Per-unit pricing

- **Review:** Preview & submit

2. Worker Acceptance

- 1. Switch to Worker role
- 2. Browse available jobs
- 3. Accept a job → View assignment
- 4. Capture evidence using in-app camera
- 5. Submit for review

3. Review Flow

- 1. Switch to Client/Correctionist role
- 2. Open Review Queue
- 3. Open SubmissionReviewPane:
 - Zoom/pan image viewer
 - OCR results panel
 - Quality metrics (edge coverage, sharpness, exposure)
- 4. Approve / Redo / Reject

4. Offline & Sync

- 1. Enable airplane mode
- 2. Complete job steps
- 3. Re-enable network
- 4. Verify sync completes

Troubleshooting

Issue	Fix
<code>adb devices</code> empty	Try different USB cable, enable USB debugging, <code>adb kill-server && adb start-server</code>
Gradle build fails	<code>./gradlew clean && ./gradlew assembleDevelopmentDebug</code>
Emulator won't start	Enable KVM: <code>ls -la /dev/kvm</code> , use <code>-gpu host</code> flag
App crashes on launch	Check <code>adb logcat -s "NetworkPeer:*</code>

Issue	Fix
API calls fail	Verify API_BASE_URL, check network security config
Camera permission denied	Settings → Apps → NetworkPeer → Permissions → Camera
iOS build fails	<pre>cd apps/ios && pod install && xcodebuild clean</pre>

Quick Reference Commands

```
# Build everything
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/openjdk.jdk/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"

# Android
cd apps/android && ./gradlew assembleDevelopmentDebug --no-daemon

# Install on device
adb install -r app/build/outputs/apk/development/debug/app-
development-debug.apk

# Logs
adb logcat -s "NetworkPeer:*" -v time

# Clear logs
adb logcat -c

# Screenshot
adb exec-out screencap -p > screenshot.png

# Screen record
adb exec-out screenrecord --output-format=h264 - > recording.mp4
```